

Email (Amazon SES)

Outbound email is scaffolded behind a transport seam that mirrors the auth provider seam: nothing in the app depends on SES directly, only on the MailTransport interface. See the [Email \(Amazon SES\)](#) section of `architecture.md`.

How it's wired

```
modules/email/
  transport.ts      MailTransport interface + EmailBody / EmailMessage / EmailFailure
  ses.ts            SesTransport – Amazon SES (v2)
  stub.ts           StubTransport – captures + logs, never sends; refuses in production
  select-transport.ts selectMailTransport(env) → stub | ses
  service.ts        sendEmail() (validates) · sendBookingConfirmation() (renders + sends)
  templates/
    booking-confirmation.tsx React Email component + renderBookingConfirmation()
```

- **EMAIL_MODE** selects the transport. It defaults to `stub`, which records messages and logs a summary but never sends. The `stub` **throws in production**, so a prod deploy that forgets `EMAIL_MODE=ses` fails loudly.
- **Failures** are typed so callers can distinguish retryable from permanent: `RATE_LIMITED` and `SERVICE_UNAVAILABLE` are transient; `EMAIL_REJECTED` (bad recipient / unverified sender / suspended account) and `VALIDATION_FAILED` (malformed message) are permanent — don't retry unchanged.
- **Templates** are React Email components rendered to `{ subject, html, text }`. Dates/times are passed in **pre-formatted** — an email has no timezone.

Status

Infrastructure + one template (`booking-confirmation`) are built and unit-tested. `sendBookingConfirmation()` is **not yet called from any route** — the booking write-flows that would trigger it (`Approved` / `Rejected` / ...) don't exist yet. When they land, a route handler does:

```
const transport = selectMailTransport(env());
if (transport) await sendBookingConfirmation(transport, user.email, input);
```

Configuring SES (production)

1. **Verify a sender identity** (a domain is recommended over a single address) in the SES console for your region, and set up DKIM.
2. **Leave the SES sandbox** (open a support request) so you can send to unverified recipients. In the sandbox, every recipient must also be verified.
3. **Create an IAM user/role** with least-privilege send permission:

```
{ "Effect": "Allow", "Action": "ses:SendEmail", "Resource": "*" }
```

(Scope Resource/add a FromAddress condition to taste.)

4. **Set the environment** (see `.env.example`):

```
EMAIL_MODE=ses
SES_REGION=ap-southeast-1
SES_ACCESS_KEY_ID=...
SES_SECRET_ACCESS_KEY=...      # keep out of the repo
EMAIL_FROM=Van Reservation <no-reply@yourdomain.com> # must be a verified identity
EMAIL_REPLY_TO=ops-support@yourdomain.com           # optional
```

EMAIL_FROM must exactly match a verified SES identity, or SES returns `MailFromDomainNotVerifiedException` → EMAIL_REJECTED.

Local development

Leave `EMAIL_MODE=stub` (the default). Sends are captured on `StubTransport.sent` and logged as `[email:stub] → ... · <subject> (stub-N)`. No AWS account or network is needed, and no real mail goes out.